DBMS Indexing: B$^+$-Tree Index Files

Introduction

Sequential index files degrade in performance with frequent insertions and deletions, requiring reorganization. The **B$^+$-tree** solves this by maintaining a balanced tree structure:

- All root-to-leaf paths have the same length.
- Efficient for both **equality queries** and **range queries**.
- Index height is kept small, minimizing disk I/O.

Key Idea

B$^+$-trees are *balanced, wide, shallow trees* optimized for disk access, supporting fast lookups, inserts, deletes, and range queries.

Structure of a B$^+$-Tree

The B$^+$-tree is defined by a parameter n, called the **order** of the tree. It determines the maximum capacity of each node:

- **Order (n):** maximum number of children that an internal node can have.
  - Each internal node can have between $\lceil n/2 \rceil$ and n children.
  - The root is an exception: it may have as few as 2 children (unless it is a leaf).
- **Keys per node:** each internal node can store up to $n-1$ **keys**.
  - Keys act as separators that guide searches.
  - Example: if an internal node has keys $K_1, K_2, \ldots, K_m$, it has $m+1$ child pointers.
- **Pointers:** Every node contains *pointers* to either child nodes (internal nodes) or actual records (leaf nodes).
  - **Internal node pointers:** An internal node of order n can have up to n pointers (and therefore up to $n-1$ keys). These pointers direct the search towards child nodes.

- **Leaf node pointers:** A leaf node of order n contains up to $n-1$ search keys, each associated with a *record pointer* that points to the actual data record (or record ID).
- Each leaf also maintains one additional `next-leaf` pointer to its right sibling leaf, enabling efficient **range queries**.

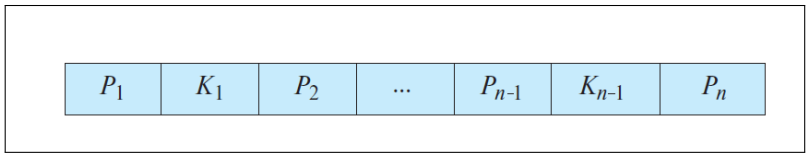

Figure 1: *Typical Node of a B^+-Tree. Each node has up to $n-1$ search keys $(K_1, K_2, \ldots, K_{n-1})$ and n pointers $(P_1, P_2, \ldots, P_n)$. Here, K means search key and P means pointer. Keys are stored in sorted order ($K_i < K_j$ if $i < j$). Pointers guide navigation: P_1 leads to values $< K_1$, P_2 to values between K_1 and K_2, and so on, with P_n covering values $\geq K_{n-1}$.*

- **Key ordering:** keys are always stored in **sorted order** within each node, ensuring fast binary or sequential search.
- **Leaves:**
  - Contain *search-key values* and corresponding *record pointers*.
  - All leaves appear at the same depth, ensuring a balanced structure.
  - Each leaf node contains between $\lceil (n-1)/2 \rceil$ and $(n-1)$ key values, where n is the maximum number of pointers (fan-out).
  - Linked together via `next-leaf` pointers to support range scans.

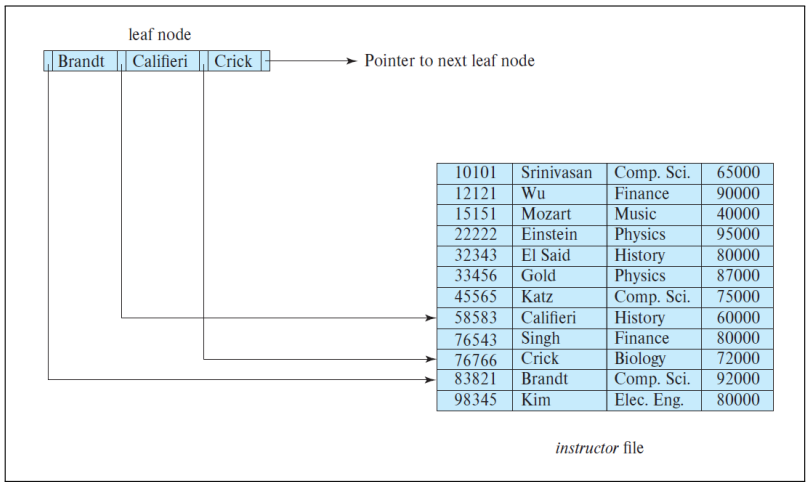

Figure 2: *A leaf node for an instructor B^+-Tree index ($n = 4$). Each leaf node stores search keys and their corresponding record pointers. Additionally, leaf nodes are linked using a `next-leaf` pointer to support efficient range queries and sequential access.*

- **Internal nodes:**
  - Do not store actual record pointers.

– Function only as a *sparse index* by routing queries to the correct leaf.

- Each internal node (except the root) contains between $\lceil n/2 \rceil$ and n child pointers, and therefore between $\lceil n/2 \rceil - 1$ and $n - 1$ keys.

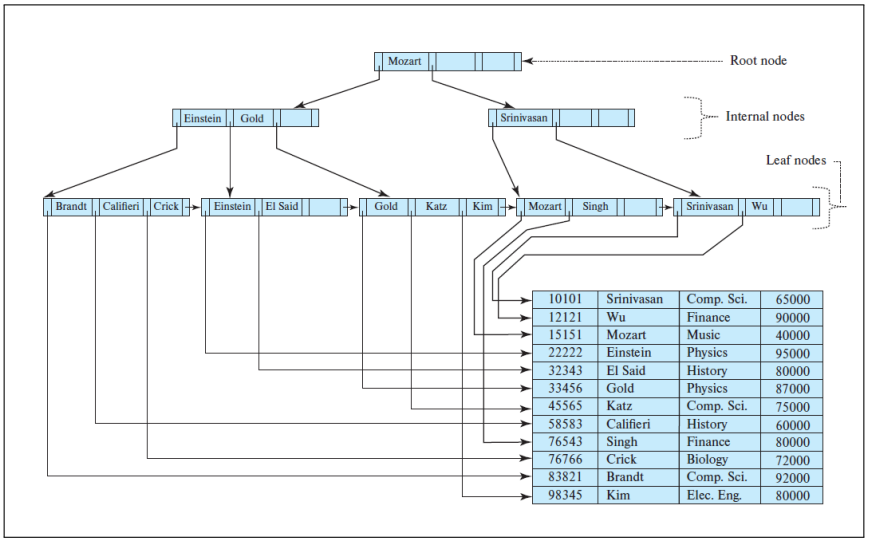

Figure 3: *Structure of a B^+-Tree Node ($n = 4$). Each internal node has at most $n = 4$ children and up to 3 keys. Leaf nodes store data pointers and are linked sequentially.*